

## Pipeline:

**Data Sources → Ingestion → Normalization → Resolution → Canonical Profile  
→ Projection → Output**

1. **Ingestion:** I will use separate readers for each source type (structured and unstructured).
  - i. **Structured Input Reader:** It will read the raw file and maps those raw fields into a dictionary. Eg: RecruiterCsvReader, ATSReader.
  - ii. **Unstructured Input Reader:** It will extract the text from the sources. We can use RegEx for parsing those data. Eg: GithubAPIReader, LinkedInProfileReader.

The output of this layer would be a raw list of unstandardized and un-normalized profile records, tagged with its source and method through which it is ingested.

2. **Normalization:** Every extracted fields will go through standard normalization functions.
  - i. **Dates:** Convert the extracted dates from string format into a unified format (YYYY-MM-DD)
  - ii. **Phone No.:** Convert the extracted phone number from string format to E.164 format.
  - iii. **Skills:** Lowercase them and strip the whitespaces.

The output of this layer would be a raw list of records but now all the values are perfectly formatted.

3. **Resolution:** 2 types of resolution will be done.
  - i. **Entity Resolution:** Matching of data from different sources will be done. For example if we get John Doe from one structured source and we get JohnDoe123 from an unstructured source (like github) then how we will know that they both are same person.  
A matching key priority list can be used:
    1. If email overlaps from both the sources -> Match
    2. If normalized phone no. overlaps -> Match
    3. If full name and date of birth overlaps -> Match
  - ii. **Conflict Resolution & Confidence Scoring:** In this layer we will merge the data extracted from different sources. If the fields extracted from different data sources conflict, then we will apply a deterministic rule for tie-breaker
    1. **Source Hierarchy:** We can preset that an official ATSReader is highly trustworthy (high confidence) than a LinkedInProfileReader and something like that for other readers.
    2. **For array fields (like skills, emails):** De-duplicate them, append them and calculate an overall confidence score based on the sources from which those are ingested.

The output of this layer would be an **Internal Canonical Profiles**.

4. **Projection:** Now the internal canonical profile will get filtered according to the user's JSON config.
  - i. It will loop through the requested fields array in the config.
  - ii. It extracts the data using the from path (eg: emails[0]).
  - iii. It will enforce the on\_missing behavior (null, omit or error).
  - iv. It will skip the confidence score if include\_confidence is false.

The final output will be the **Master Canonical Profile** based on user's config.

## Canonical Profile Schema & Normalized Formats:

1. Candidate\_id (string)
2. full\_name (string)
3. headline (string or null)
4. years\_of\_experience (number or null)
5. emails (string list)
6. skills (string list)
7. phones (string list in E.164 format)
8. location ({city, region, country} in string)
9. date\_of\_birth (YYYY-MM-DD format)
10. provenance ({field: string, source: string, method: string}) ; where source is the name of the source from where it is extracted and method is the type of reader used for extraction.

## Edge Cases:

1. **Same Name but different persons Conflict:** When 2 different sources gets same data of any field (let say same full\_names). **Solution:** Multi-key matching algorithm should be used and if any of the other field/key is not overlapping then they must be treated as separate profiles.
2. **Garbage Normalised Data:** If any field such as phone number has a data which cannot be parsed into E.164 or any different standard format, let say phone number reads "No calls on Sunday". **Solution:** Normalization should be robust. If a value cannot be matched via Regex or any other standard parsing technique, then it should be marked as **null** value.
3. **Out-of-bound location:** If the custom config file requests an array path selection which does not exists or is invalid. **Solution:** Projection layer must use try-catch block or any other fallback block. It then immediately triggers to the user defined "on\_missing" policy (null, omit or error).
4. **Conflicting channels:** If two different sources projects different data for a same profile. Let say for John Doe, source A projects different email and source B projects different email. **Solution:** Must retain both the unique emails in the internal canonical emails list.
5. **Domain Specific Semantic Clustering:** Ingesting disparate resume skills like "AWS" from source A and "Amazon Web Services" from source B. **Solution:** They must be treated same.

## Things deliberately left out due to time constraint:

1. Limit the use of LLM based or any other heavy PDF Parser due to time constraint
2. The complex mapping (Edge Case 5) is descoped for now and will be treated unique.

